Project PRD (Dayananda Sagar University)

Sep 30, 2026 · @Anoop

1. Overview

Project: AI-Powered Pothole Detection and Road Accountability System. Built by Anoop Pola (ENG23DS0003) and Amrutalakshmi H K (ENG24DS3001), Dept. of CSE (Data Science), Dayananda Sagar University. Supervised by Prof. Manjula M.

One-line pitch: Turn ordinary dashcam or phone road video into a list of *unique*, geo-tagged, severity-scored pothole events. Each event carries an evidence record, is routed to the responsible authority, tracked to repair and verified by re-inspection.

Problem statement (from the PPT): India recorded 23,056 pothole-related accidents and 9,438 deaths in 2020–2024 (MoRTH data tabled in Parliament). Of Bengaluru's 15,403 pothole complaints (Apr 1 to Dec 2, 2024), about 31% were rejected as duplicates and about 15% stayed open. Existing research detects potholes frame by frame. It does not answer three questions:

- Is this the *same* pothole I saw 2 frames ago? This is the event-level association gap.
- How bad is it, when a 2D image has no depth? This is the severity gap.
- Who must fix it, and was it actually fixed? This is the evidence and accountability gap.

What makes this project different: the novelty is not the detector. YOLOv8-Seg is off the shelf. The contribution is the pipeline *after* detection: temporal association and deduplication into unique events, a segmentation-based Visual Severity Proxy, and a before/after re-inspection loop. The PRD protects that focus: detector work is kept just good enough, and most effort goes into the event, severity and accountability layers.

Current status: 20% at Review 1. Dataset study and preprocessing are done. YOLOv8-Seg setup, training preparation and initial testing are in progress.

2. Objectives, scope and success criteria

The project succeeds if one uploaded road video produces a deduplicated, severity-ranked pothole list on a map, with each event backed by an evidence card. A second video of the same road must show which potholes were repaired.

Objectives (mapped to the research questions):

1. **O1 Detection and segmentation:** fine-tune YOLOv8-Seg to detect and mask potholes in road frames.

2. **O2 Unique events (RQ1):** associate repeated detections across frames and GPS positions into one event per physical pothole.
3. **O3 Visual Severity Proxy (RQ2):** compute a Low / Medium / High score from mask-derived features. Validate it against human severity labels.
4. **O4 Robustness (RQ3):** measure how accuracy changes with lighting, motion blur, vehicle speed and road type.
5. **O5 Accountability:** generate evidence cards, route each event to NHAI / PWD / municipal body, track status and show a public dashboard.
6. **O6 Re-inspection:** match events across two drives of the same road. Classify each as Repaired, Still present or Worsened.

In scope: offline processing of uploaded videos, one class (pothole), a web dashboard, a simulated authority workflow, and Bengaluru roads for field data.

Out of scope: filing real complaints on BBMP Sahaaya or other live portals (the system produces a ready-to-file complaint package instead), real-time on-vehicle inference, true physical depth measurement (no LiDAR or stereo), other road defects (cracks, rutting) and a native mobile app.

Success criteria (targets to confirm with your guide):

Area	Metric	Target
Segmentation	Mask mAP@0.5 on held-out test set	⩾ 0.75 (Padhi et al. report 0.805)
Unique events	Event-level F1 on annotated test videos	⩾ 0.80
Deduplication	Duplicate rate (extra events per real pothole)	⩽ 10%
Severity	Agreement with human labels (weighted Cohen's κ)	⩾ 0.6
Re-inspection	Repair-status accuracy on before/after pairs	⩾ 80%
Speed	End-to-end processing on a laptop GPU / Colab T4	⩾ 5 frames/s at 1 FPS sampling

3. Literature review: what we borrow from each paper

The 10 papers in your PPT show detection is largely solved (mAP50 around 0.77–0.82). None of them links detections into tracked, verifiable events. That gap is exactly this project.

Papers cited in the PPT

Paper	Venue, year	What we take into the build
Padhi et al., Pothole Detection and Instance Segmentation Using YOLOv8	IEEE Conf., 2024	Baseline to beat: YOLOv8m-seg, Box mAP50 82.1%, Mask mAP50 80.5% on 3,074 images. Use as our segmentation benchmark.
Cano-Ortiz et al., End-to-End CV System for Pavement Distress Detection and Quantification	Constr. & Building Materials, 2024	Rule-based post-processing cut false positives by 20.5%. Copy the idea: filter masks by size, position (road region only) and track length.
de Souza et al., Classification of Pothole Distress Severity Using YOLOv8	DYNA, 2025	Low / Medium / High labels on 2,618 images. Low and medium were hardest, and missing depth was the stated cause. Justifies a *proxy* score validated by humans.
Zhong et al., YOLOv8 and Point Cloud Fusion	Scientific Reports, 2025	Real depth needs RGB-D. We state this as a limitation and do not claim physical depth.
Bamitale et al., Road Safety for Everyone	IEEE ICCE, 2026	GPS + timestamp + Haversine distance + spatial clustering, with LLM summaries. Basis of our GPS deduplication step.
Bhatia et al., Real-Time Pothole Detection Using YOLO Models	IJTST, 2026	YOLO11n: 1.1 ms inference, 5.6 MiB. Supports an optional YOLO11-seg ablation.
Padala et al., Phase Congruency Enhanced YOLO on Edge Device	IEEE Access, 2026	Edge deployment on Raspberry Pi 5. Future work, not in scope.

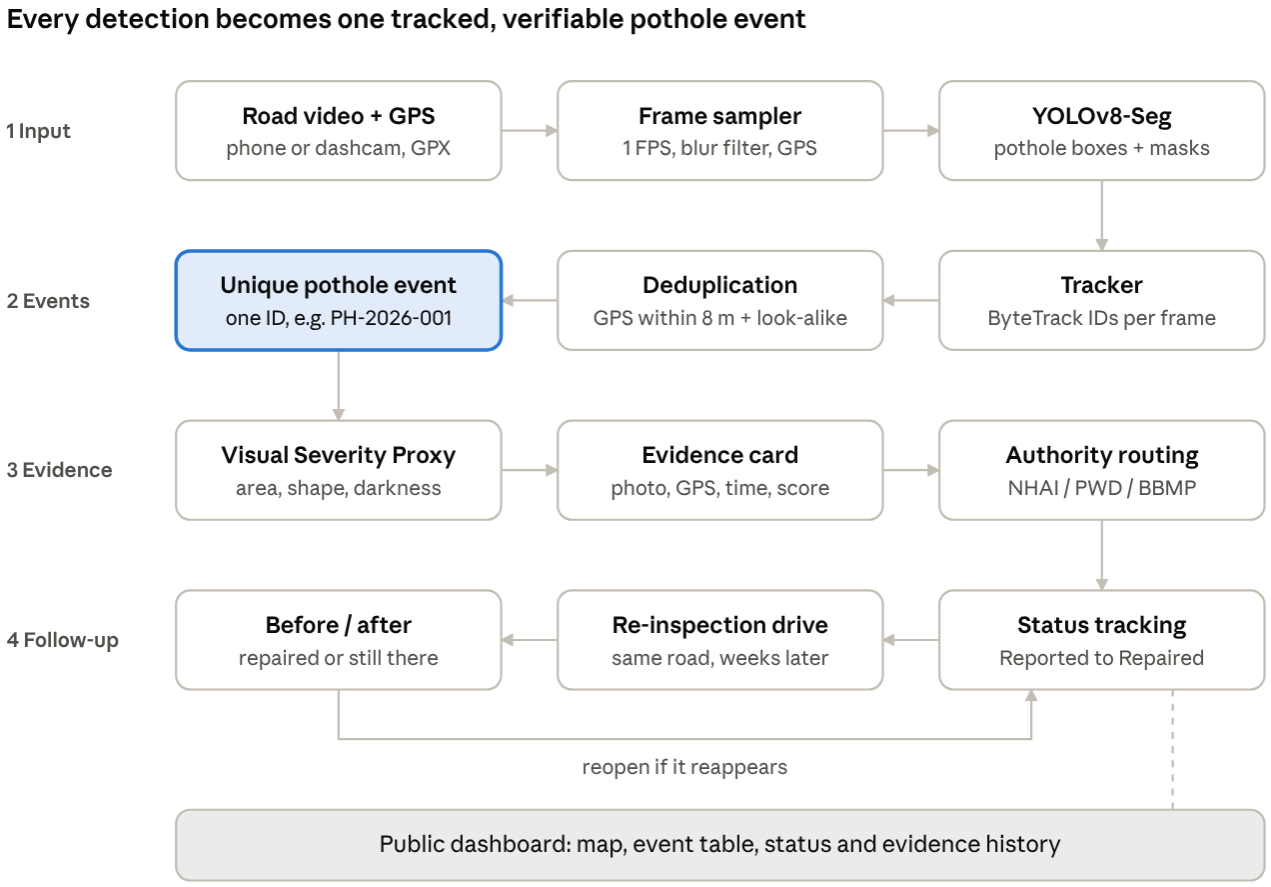
Paper	Venue, year	What we take into the build
Yang et al., Semi-Supervised YOLOv8 Network	Measurement, 2026	Teacher–student pseudo-labels. Optional if our labelled data is small.
Road Surface Defect Detection Algorithm Based on YOLOv8	Electronics, 2024	77.4% mAP@0.5 on RDD2022, and weather/lighting robustness still weak. Motivates RQ3.
Kundaliya et al., Smart Pot Hole Detection System	PNAS India Sec. A, 2026	Accelerometer-based alternative. Cite as a contrast only.
YOLOv8-PD	Scientific Reports, 2024	Listed in references [2]. Add a row to the SOTA slides for it.

Outside references to add (standard methods the build uses; verify citations before the report)

Reference	Why it matters
Zhang et al., ByteTrack, ECCV 2022	Multi-object tracker built into Ultralytics (`model.track`). Gives frame-to-frame IDs for Temporal Association.
Aharon et al., BoT-SORT, 2022	Alternative tracker with camera-motion compensation, which helps with a moving dashcam. Compare it with ByteTrack.
Arya et al., RDD2022 dataset, 2022	Main training data. **Its labels are bounding boxes only (D40 = pothole), not masks.** See Section 6 for how we get masks.
Kirillov et al., Segment Anything (SAM), ICCV 2023	Converts RDD2022 pothole boxes into masks (box-prompted), saving manual polygon labelling.
Yang et al., Depth Anything V2, NeurIPS 2024	Optional monocular *relative* depth cue for the severity proxy. Experimental, clearly labelled as relative, not metric.
Ultralytics YOLOv8 docs	Training, segmentation export and tracking API used throughout.

4. Proposed system and architecture

The system is one pipeline in 4 stages, matching slides 4 and 12 of your PPT. The accented box, the unique pothole event, is the unit everything after it works on.



system architecture · 4 stages, 12 modules

The flow snakes left to right, then right to left, row by row. Re-inspection loops back to status tracking, so a pothole that reappears is reopened automatically.

Core data model: `Event` (id, GPS, road, authority, severity, status) has many `Observation`s (frame, mask, confidence, video, time) and many `StatusChange`s (from, to, time, note). An `Inspection` is one processed video, and it links the observations it produced.

5. Requirements

There are 14 functional requirements. The P0 items are the minimum demo for the final review; P1 items complete the accountability story; P2 items are stretch goals.

Functional requirements

ID	Requirement	Priority
FR1	Upload a road video (MP4, up to 10 min) with an optional GPS track (GPX/CSV from a phone logger app). Without GPS, the user enters a start point and route.	P0
FR2	Sample frames at about 1 FPS, configurable 1–5 FPS. Discard blurred frames (variance-of-Laplacian threshold) and log how many were dropped.	P0
FR3	Run YOLOv8-Seg on each frame. Output box, mask, confidence and frame timestamp.	P0
FR4	Mask to the lower road region (region of interest) and drop tiny or low-confidence masks to reduce false positives.	P0
FR5	Link detections across frames with ByteTrack / BoT-SORT into tracks. Drop tracks seen in fewer than N frames (default 2).	P0
FR6	Deduplicate tracks into unique pothole events. Merge tracks within R metres (default 8 m, Haversine) that are visually similar. Assign an ID such as PH-2026-001.	P0
FR7	For each event, pick the best frame (highest confidence x mask area) and compute severity features and the Low / Medium / High Visual Severity Proxy.	P0
FR8	Generate an evidence card: best frame with mask overlay, crop, timestamp, GPS, address, severity and feature values. Export as PDF/PNG and JSON.	P0
FR9	Map each event to the responsible authority from road type: National Highway to NHAI, State Highway to PWD, otherwise the municipal body (BBMP in Bengaluru). Use OpenStreetMap road tags.	P1
FR10	Track status as Reported, Assigned, Repair in Progress, Repair Reported, Verified Repaired or Reopened, with a timestamped history.	P1

ID	Requirement	Priority
FR11	Public dashboard: map with severity-coloured pins, filterable event table and event detail page with evidence history.	P1
FR12	Re-inspection: process a second video of the same road and match new events to old ones by location. Mark each old event Repaired, Still present or Worsened, and flag new potholes.	P1
FR13	Generate a before/after comparison image for each re-inspected event. Auto-reopen when a repaired pothole reappears.	P1
FR14	Ready-to-file complaint package (text + evidence) formatted for BBMP Sahaaya. Also export the whole report as CSV.	P2

Non-functional requirements

- **Performance:** a 5-minute video processes in under 5 minutes on a Colab T4 or laptop GPU.
- **Reproducibility:** fixed random seeds, pinned `requirements.txt`, and every experiment logged with its config and weights.
- **Explainability:** every severity score shows the feature values behind it. No black-box severity numbers.
- **Honesty of claims:** the UI and report always say *Estimated Visual Severity Proxy* and never state depth in cm.
- **Privacy:** blur faces and number plates on evidence images before storing or showing them, e.g. with a small face/plate detector or Gaussian blur on detected persons and vehicles.
- **Usability:** the full flow from upload to processing, map and evidence card takes no more than 3 clicks.

6. Tech stack, datasets and hardware

The stack keeps your PPT's choices (Python, YOLOv8-Seg, OpenCV, NumPy/Pandas, Jupyter, Git) and adds only what the accountability layer needs.

Tech stack

Layer	Choice	Why
Detection / segmentation	Ultralytics YOLOv8-Seg (`yolov8s-seg` to start, `yolov8m-seg` if the GPU allows)	As proposed in the PPT, with built-in training, export and tracking
Tracking	ByteTrack and BoT-SORT via `model.track()`	Frame-to-frame IDs without writing a tracker from scratch
Video / image	OpenCV, FFmpeg	Frame sampling, blur check, overlays
Label help	SAM (box-prompted), Roboflow or CVAT	Masks from boxes, then manual correction
Geo	`gpxpy` , `haversine` , OSM Nominatim (reverse geocode), Overpass API (road type)	Location, address and authority routing
Backend	FastAPI + SQLite (upgrade to PostgreSQL + PostGIS only if needed)	Simple REST API; SQLite is enough for a demo
Dashboard	Streamlit with Folium/Leaflet map, or React + Leaflet if time allows	Streamlit is quickest to build and fine for a university demo
Reports	Jinja2 + WeasyPrint (PDF evidence cards)	Printable evidence
Experiments	Jupyter, Weights & Biases or CSV logs, Git/GitHub	Reproducibility
Compute	Google Colab (T4) or Kaggle (P100/T4) for training; any laptop for the app	Free GPU

Datasets

Dataset	Use	Note
RDD2022 (D40 pothole class, India + Japan subsets)	Training and validation	Boxes only. Convert to masks with SAM, then spot-check about 10%.

Dataset	Use	Note
Public pothole *segmentation* sets (Kaggle / Roboflow Universe, YOLOv8-seg format)	Training with ready masks	Check licence and remove duplicates of RDD images.
Own Bengaluru road videos (phone on dashboard + GPS logger app)	Test set only, never used for training	Aim for 10–15 clips of about 5 min: day/dusk/night, dry/wet, 20–60 km/h.
Re-inspection pairs	Before/after evaluation	The same 3–5 routes filmed twice, weeks apart. Include at least one road that has been repaired.
"RDD2024" (listed in the PPT)	To confirm	I could not confirm a public dataset by this name. Check the source or remove it from the slide.

Ground truth you must annotate yourselves: for each test video, list every real pothole with a GPS point and a Low/Medium/High label from 2 annotators. This is the only way to measure event-level F1 (RQ1) and severity agreement (RQ2). Budget about 1 day per 10 clips.

Hardware: Android or iPhone with a phone mount, a GPS logger app (e.g. GPS Logger or Geo Tracker, exporting GPX), and a Colab or Kaggle account. Optionally a measuring tape and ruler to physically measure about 20 potholes for severity validation.

7. Step-by-step implementation plan

The build runs in 7 phases. Each phase ends with something you can demo, so a slip in a later phase never leaves you with nothing to show.

1. **Phase 1: Data and labels (finish the current 20%)**
   - Download RDD2022 and keep the D40 (pothole) images. Add 1–2 public pothole segmentation sets.
   - Convert boxes to masks with box-prompted SAM, then review about 10% by hand in CVAT or Roboflow.
   - Merge the sets into YOLOv8-seg format with a 70/15/15 split, deduplicated by image hash.
   - *Deliverable:* `data/pothole_seg.yaml` and a dataset statistics table for the report.
2. **Phase 2: Train and benchmark YOLOv8-Seg (O1)**

- Train `yolov8s-seg` at 640 px for about 100 epochs with default augmentation plus brightness, blur and rain augmentation.
- Compare with `yolov8m-seg`, plus `yolo11s-seg` as an optional ablation. Record Box/Mask mAP@0.5, precision, recall and FPS.
- *Deliverable:* `best.pt` and a results table next to Padhi et al.'s 80.5% mask mAP50.

3. **Phase 3: Video pipeline and unique events (O2, RQ1), the core novelty**
   - Sample frames at 1 FPS and skip blurred frames. Align each frame with GPS by timestamp, interpolating between GPS points.
   - Run `model.track(tracker='bytetrack.yaml')` for track IDs. Keep tracks seen in 2 or more frames.
   - Deduplicate across tracks. Merge two tracks when their GPS points are within 8 m (Haversine) *and* their best crops look similar (e.g. HSV colour histogram or a small CNN embedding, cosine similarity > threshold).
   - Compare 3 variants: detections only, tracker only, and tracker + GPS deduplication. This comparison answers RQ1.
   - *Deliverable:* `events.json` with one entry per pothole.
4. **Phase 4: Visual Severity Proxy (O3, RQ2)**
   - From the best mask, compute: area as a share of the road region; perspective-corrected area, rescaling pixels by row position so far potholes are not under-counted (e.g. a bird's-eye homography calibrated once per phone mount); elongation and solidity (shape irregularity); dark-interior ratio (shadow or depth cue); number of frames the pothole persisted.
   - Combine them into one score (formula below). Set the Low/Medium thresholds from percentiles, then tune them on the human-labelled set.
   - Optional: add relative depth from Depth Anything V2 as a feature and report whether it improves agreement.
   - *Deliverable:* a severity module plus a correlation and confusion-matrix report.
5. **Phase 5: Evidence, routing and status (O5)**
   - Build the evidence card (Section 5, FR8). Blur faces and plates.
   - Reverse-geocode each event and look up the OSM road tag to route it to NHAI, PWD or BBMP.
   - Build a SQLite schema with `events`, `observations`, `status_history` and `inspections`, and FastAPI endpoints for upload, events and status updates.
6. **Phase 6: Dashboard and re-inspection (O6)**
   - Build a Streamlit dashboard with a map, filters, an event page and a status change control for the admin role.

- Re-inspection: process the new video and match its events to open events within 8 m. If no pothole is detected where one was, and the frame is good quality and the road is visible, mark it *Repaired (pending verification)*. If the mask area grew more than 30%, mark it *Worsened*.
- Generate a side-by-side before/after image for each matched event.

7. **Phase 7: Evaluation, robustness and write-up (RQ3)**
   - Split test clips by condition (day/dusk/night, dry/wet, speed band). Add synthetic motion blur and low-light variants.
   - Report every metric per condition. Write the report and paper draft, and record a 3-minute demo video.

Severity proxy formula (starting point, weights tuned in Phase 4):

```
S = 0.45\,\hat{A} + 0.20\,\hat{D} + 0.15\,(1 - \mathrm{solidity}) +
0.20\,\hat{P}
```

Here Â is the perspective-corrected area, D̂ the dark-interior ratio and P̂ the persistence across frames, each scaled to 0–1. The first cut is Low if S < 0.33, High if S ⩾ 0.66, otherwise Medium.

Suggested repository layout

```
pothole-accountability/
  data/            # dataset yaml, split lists (no raw data in git)
  notebooks/       # EDA, training, experiments
  src/
    detect/        # YOLOv8-Seg inference wrapper
    track/         # tracker + GPS dedup -> events
    severity/      # features + proxy score
    geo/           # GPX sync, reverse geocode, authority routing
    evidence/      # evidence cards, blur, PDF
    reinspect/     # before/after matching
  api/             # FastAPI app + SQLite models
  dashboard/       # Streamlit app
  eval/            # metric scripts, ground-truth files
  reports/         # figures and tables for report / paper
```

8. Evaluation plan

Each research question gets its own experiment, metric and table in the report. The PPT already lists the metrics; this section says how each one is measured.

Question	Experiment	Metrics	Expected result to report
Detector (O1)	Test split of the merged dataset; yolov8s-seg vs yolov8m-seg (vs yolo11s-seg)	Precision, Recall, Box and Mask mAP@0.5, Mask IoU, FPS	Mask mAP@0.5 close to Padhi et al.'s 80.5%
RQ1: unique events	Own test videos with hand-listed ground-truth potholes; compare detections only, tracker only, and tracker + GPS dedup	Event precision, event recall, event F1, duplicate rate, optional IDF1	Tracker + GPS dedup has the lowest duplicate rate at similar recall
RQ2: severity	Proxy label vs majority label of 2 human annotators; optional tape-measured size on about 20 potholes	Weighted Cohen's κ, confusion matrix, Spearman ρ (score vs measured area)	Low vs High separates well; Medium is the hardest, consistent with de Souza et al.
RQ3: robustness	Same metrics split by lighting, wet/dry, speed band and synthetic blur level	Event F1 and Mask mAP per condition, plus the drop from baseline	A chart of the performance drop per condition
Re-inspection (O6)	Before/after pairs with known outcomes	Repair-status accuracy, false "Repaired" rate	False "Repaired" rate is the key safety number and should be low
System	5-min video end to end	Total processing time, frames/s	Under 5 min on a T4

How an event counts as a match: a predicted event is a true positive if it lies within 8 m of a ground-truth pothole that has not already been matched. Extra predictions near an already-matched pothole count as duplicates.

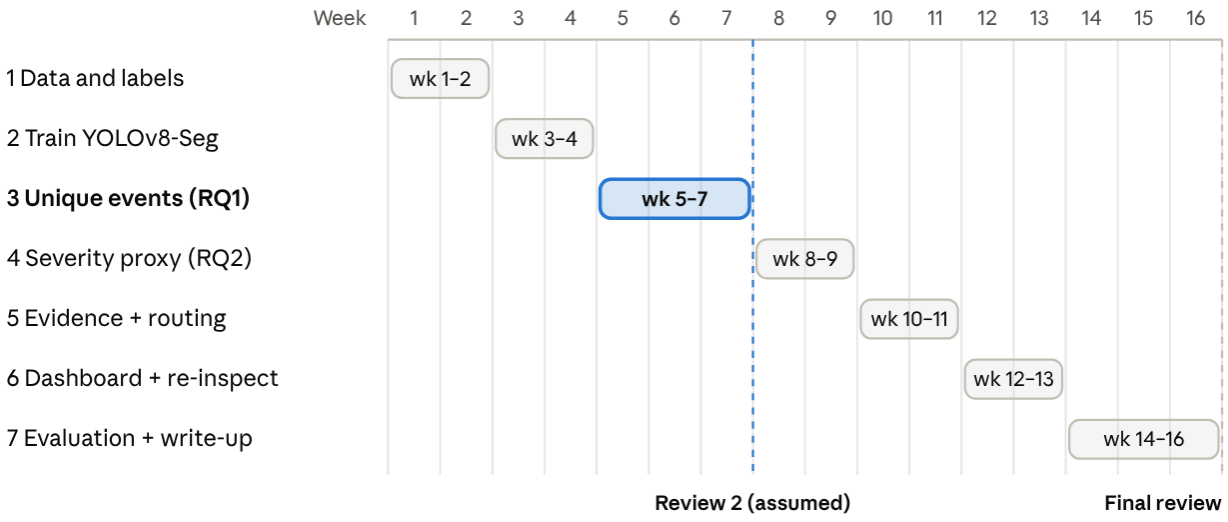
Test plan for the software: unit tests for the Haversine merge, severity feature functions and authority routing (`pytest`). Add one end-to-end test that runs a 20-second sample clip and checks that `events.json` is produced.

9. Timeline, deliverables and risks

The plan fits in about 16 weeks from now. The final dates depend on your review calendar, which I don't have yet.

The core event pipeline is ready by week 7; write-up starts week 14

Build plan by week from project restart; review dates to be confirmed



build roadmap · 7 phases over 16 weeks

Phase 3 is the project's novelty, so it gets the most time. Start filming test videos in weeks 1–2, alongside Phase 1, so you have real clips by Phase 3.

Deliverables

- [ ] GitHub repository with README, `requirements.txt` and trained weights
- [ ] Trained YOLOv8-Seg model and benchmark table
- [ ] Event pipeline (`events.json`), severity module and evidence cards (PDF)
- [ ] Streamlit dashboard with map, status tracking and re-inspection view
- [ ] Annotated test-video ground truth (reusable dataset contribution)
- [ ] Project report, review PPTs, 3-minute demo video
- [ ] Paper draft (target: an IEEE conference or a journal like those in your SOTA table)

Risks and fallbacks

Risk	Impact	Fallback
Few mask labels (RDD2022 is boxes only)	Weak segmentation	SAM box-to-mask conversion plus public seg sets. Semi-supervised pseudo-labels (Yang et al.) if still short.

Risk	Impact	Fallback
Phone GPS noise of 5–15 m	Wrong merges or splits in deduplication	Tune the merge radius per clip. Combine GPS with visual similarity, not GPS alone.
No repaired road to film before the deadline	Re-inspection cannot be validated	Scout a pothole reported to BBMP now and film it again later. Otherwise use a public before/after image set for a lab test.
Severity proxy disagrees with humans	RQ2 result is weak	Report it honestly. A negative or partial result is still a valid finding. Try the relative-depth feature.
Night / rain clips perform badly	RQ3 looks poor	That is the finding RQ3 asks for. Report it and add augmentation as mitigation.
Scope creep on the dashboard	Core ML slips	Streamlit only. Build React only after Phase 7 is done.

Open questions for you and your guide

- What are the dates of Review 2, Review 3 and final submission?
- Is "RDD2024" a specific dataset you found? If so, where is it from?
- Do you have a GPU laptop, or will you use Colab / Kaggle only?
- Should the paper target a conference (faster) or a journal?